UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
Facultad de Ciencias de la Computación
Carrera de Software

Fichas bibliográficas

Fundamentos de sistemas distribuidos
y arquitecturas distribuidas

Replicación y tolerancia a fallos en Apache Kafka para la comunicación entre microservicios

Estudiante	Coronel Gómez Khriz Alexander
Asignatura	Aplicaciones Distribuidas (ISR-701)
Nivel y paralelo	Séptimo, A
Actividad	FORM-FB-01, formativa individual
Periodo de la actividad	Semana 2, del 2 al 8 de noviembre de 2026

Enfoque para TA-IND-01

£Cómo influyen la replicación y la configuración de Apache Kafka en la conservación de mensajes, la disponibilidad y la recuperación ante fallos? Las fuentes sobre Kafka Streams permiten examinar aplicaciones que procesan eventos. Sus resultados se atribuyen a los componentes y versiones estudiados.

Estado de revisión

Se incluyen tres artículos de revista y dos de congresos ACM. Continúan pendientes la lectura integral de las dos primeras fuentes y la evidencia directa de indexación indicada en el README.

Repositorio del trabajo

`https://github.com/ereinosov/app-dist-form-fb-01`

FICHA 1 / 5

Fundamentos y líneas de investigación

Referencia IEEE

T. P. Raptis and A. Passarella, «A survey on networked data streaming with Apache Kafka», *IEEE Access*, vol. 11, pp. 85 333–85 350, 2023

Identificador DOI

`https://doi.org/10.1109/ACCESS.2023.3303810`

Resumen de 100 palabras

Raptis y Passarella estudian la dispersión de investigaciones sobre transmisión de datos mediante Apache Kafka. Organizan la literatura en cinco áreas, algoritmos, redes, datos, sistemas ciberfísicos y seguridad. Su método consiste en revisar y clasificar trabajos académicos, relacionando las propuestas de optimización con aplicaciones concretas. El resultado es una visión organizada de los problemas investigados y de las oportunidades que siguen abiertas. La revisión permite situar Kafka dentro de sistemas que intercambian información continuamente y reconocer que su evaluación requiere considerar varias dimensiones. Sirve como punto de partida para seleccionar aspectos de arquitectura, rendimiento y fiabilidad que merecen análisis posterior.

Dato concreto y localización

El artículo organiza la literatura en cinco áreas, algoritmos, redes, datos, sistemas ciberfísicos y seguridad. Ubicación en el abstract, p. 85333.

Nota para TA-IND-01

Esta clasificación me ayuda a ubicar la tolerancia a fallos dentro de un panorama más amplio. Usaría la fuente en el marco teórico de TA-IND-01 para presentar Kafka y delimitar las dimensiones que analizaré.

Verificación y versión consultada

Se cotejaron metadatos y el abstract de la publicación. La lectura integral del PDF permanece pendiente.

FICHA 2 / 5

Particionado y configuración del clúster

Referencia IEEE

T. P. Raptis, C. Cicconetti, and A. Passarella, «Efficient topic partitioning of Apache Kafka for high-reliability real-time data streaming applications», *Future Generation Computer Systems*, vol. 154, pp. 173–188, May 2024

Identificador DOI

`https://doi.org/10.1016/j.future.2023.12.028`

Resumen de 100 palabras

El artículo aborda la dificultad de configurar particiones y brokers de Apache Kafka según las necesidades de aplicaciones que intercambian datos en tiempo real. Los autores proponen optimizar ambos parámetros considerando características y restricciones del escenario estudiado. Evalúan su propuesta mediante simulaciones numéricas y experimentos con un prototipo sobre un clúster Kafka. La evaluación incluye tres escenarios relacionados con producción, consumo y transmisión en tiempo real. Su aporte principal es formular la configuración como una decisión dependiente de requisitos, en lugar de asumir valores universales. Este enfoque permite examinar cómo el particionado condiciona el funcionamiento de una arquitectura distribuida concreta.

Dato concreto y localización

La evaluación incluye tres escenarios de producción, consumo y transmisión de datos en tiempo real. Ubicación en el abstract del artículo, p. 173.

Nota para TA-IND-01

Me interesa relacionar el número de particiones con las necesidades de una aplicación. Incorporaría esta fuente en el análisis de arquitectura y configuración, explicando qué requisitos justifican las decisiones del clúster.

Verificación y versión consultada

Se cotejaron metadatos y el abstract editorial. La lectura integral del PDF permanece pendiente.

FICHA 3 / 5

Escalabilidad de microservicios

Referencia IEEE

S. Henning and W. Hasselbring, «Benchmarking scalability of stream processing frameworks deployed as microservices in the cloud», *Journal of Systems and Software*, vol. 208, 111879, Feb. 2024

Identificador DOI

`https://doi.org/10.1016/j.jss.2023.111879`

Resumen de 100 palabras

Henning y Hasselbring investigan la escalabilidad de herramientas de procesamiento continuo utilizadas dentro de microservicios. Comparan cinco frameworks mediante experimentos sistemáticos ejecutados en Kubernetes, tanto en una nube pública como en infraestructura privada. Analizan cómo cambian los recursos necesarios cuando aumenta la carga o el trabajo computacional. Los resultados muestran escalabilidad aproximadamente lineal cuando existen recursos suficientes, aunque la cantidad requerida varía considerablemente entre herramientas y casos de uso. Ningún framework resulta superior en todos los escenarios. La investigación aporta un procedimiento reproducible para evaluar escalabilidad y permite relacionar las decisiones de despliegue con los costos operativos de aplicaciones distribuidas.

Dato concreto y localización

El estudio informa más de 740 horas de experimentos, con hasta 110 instancias simultáneas de microservicios. Ubicación en el abstract, apartado Method, p. 1 del manuscrito consultado.

Nota para TA-IND-01

Esta fuente me orienta para definir qué significa escalar y qué recursos debo observar. La usaría en la evaluación de escalabilidad de TA-IND-01, considerando el costo de sostener una carga creciente.

Verificación y versión consultada

Se consultó la versión de los autores en arXiv, identificador 2303.11088. La referencia corresponde al artículo de revista de 2024.

FICHA 4 / 5

Recuperación y estabilidad ante fallos

Referencia IEEE

A. Vogel, S. Henning, E. Perez-Wohlfeil, O. Ertl, and R. Rabiser, «A comprehensive benchmarking analysis of fault recovery in stream processing frameworks», in *Proceedings of the 18th ACM International Conference on Distributed and Event-based Systems*, Association for Computing Machinery, 2024, pp. 171–182

Identificador DOI

`https://doi.org/10.1145/3629104.3666040`

Resumen de 100 palabras

Vogel y sus colaboradores examinan cómo se recuperan Flink, Kafka Streams y Spark Structured Streaming cuando fallan sus trabajadores. Utilizan ShuffleBench sobre Kubernetes e inyectan fallos recurrentes mediante técnicas de ingeniería del caos. Miden rendimiento, latencia, estabilidad y tiempo de recuperación bajo distintos escenarios. Sus resultados muestran diferencias entre frameworks y evidencian que recuperar la ejecución puede acompañarse de degradaciones persistentes. En Kafka Streams observan problemas relacionados con el rebalanceo y la distribución desigual del trabajo después de fallos. El estudio ofrece un método experimental para evaluar resiliencia, considerando tanto la continuidad del procesamiento como el comportamiento posterior del sistema.

Dato concreto y localización

En el escenario de eliminación de pods se interrumpen 1, 2 o 4 trabajadores simultáneamente, con fallos repetidos cada cinco minutos. Ubicación en la sección 5.2, Failure Injection Scenarios, de la versión de los autores.

Nota para TA-IND-01

Me permite distinguir la recuperación del servicio de la recuperación de su rendimiento. Usaría los escenarios y métricas en el análisis crítico de tolerancia a fallos, manteniendo explícita la versión de Kafka Streams evaluada.

Verificación y versión consultada

Se consultó la versión de los autores en arXiv, identificador 2404.06203v3. El estudio emplea Kafka Streams 3.5, según su sección 4.2.

FICHA 5 / 5

Ajuste de parámetros y rendimiento

Referencia IEEE

D. Chen, S. Henning, K. Matteussi, and R. Rabiser, «Performance optimization in stream processing systems: Experiment-driven configuration tuning for Kafka Streams», in *Companion of the 17th ACM/SPEC International Conference on Performance Engineering*, Association for Computing Machinery, 2026, pp. 154–159

Identificador DOI

`https://doi.org/10.1145/3777911.3800636`

Resumen de 100 palabras

Chen y sus colaboradores investigan cómo mejorar el rendimiento de Kafka Streams mediante ajustes de configuración. Combinan muestreo por hipercubo latino, recocido simulado y búsqueda local, integrados con Theodolite para ejecutar experimentos en Kubernetes. Evalúan configuraciones sobre ShuffleBench y detienen anticipadamente aquellas con resultados deficientes. Encuentran mejoras de hasta el veintitrés por ciento frente a los valores predeterminados. El refinamiento final aporta beneficios limitados dentro de la variabilidad experimental. También observan que las configuraciones con mayor rendimiento incrementan la latencia. Sus resultados permiten discutir compromisos de configuración, aunque corresponden a una evaluación piloto con condiciones específicas de infraestructura y carga.

Dato concreto y localización

La mejor ejecución de recocido simulado mejora aproximadamente un 23 % el throughput frente a la configuración base. Ubicación en la sección 4.2.2, Simulated Annealing, de la versión de los autores.

Nota para TA-IND-01

La fuente me ayuda a cuestionar el uso automático de valores predeterminados. La emplearía en la discusión de configuración de TA-IND-01, relacionando rendimiento y latencia con las condiciones del experimento.

Verificación y versión consultada

Se consultó la versión de los autores en arXiv, identificador 2603.04027v1. La sección 4.3 delimita los resultados a una evaluación piloto.